

Reflection

Reproducible NLP Pipeline in a Container — Slim Version

01 Which approach won, and what does that tell you about when a classical method is still worth using?

The classical baseline won decisively: **0.967 accuracy / 0.920 macro F1 in 0.12 s** versus the transformer's **0.450 / 0.426 in 137 s**. The transformer is over 1,100× slower for an outcome worse than the trivial "always predict ham" baseline (~0.866 on this class distribution).

The reason is not that DistilBERT is bad — it is that I deliberately used a sentiment model (`distilbert-base-uncased-finetuned-sst-2-english`, trained on movie reviews) and remapped `POSITIVE` → ham, `NEGATIVE` → spam. That mapping presumes "positive in tone" correlates with "legitimate message," which is a guess, not a property of the data. A cheerful spam ("Congratulations! You won a free iPhone!") reads as `POSITIVE` and gets labelled ham; a terse legitimate message ("ok") reads as `NEGATIVE` and gets labelled spam. The classifier is solving the wrong problem.

The classical baseline does not care about tone. It learns the words ("free", "txt", "claim", "won") that distinguish spam from ham *in this corpus*, fit in a fraction of a second on a CPU. When the task is well-specified, the labels are clean, and a few thousand examples are available, TF-IDF + linear classifier is still the right answer most of the time. Reach for a pretrained transformer when there is genuine linguistic complexity (sarcasm, long context, paraphrasing) **and** the pretraining objective matches your task — or when you are willing to fine-tune. Zero-shot on a mismatched head is a trap.

02 What's in your Docker image that wouldn't be in a venv, and why does that matter for someone trying to reproduce your work?

Three things the image pins that a `venv` cannot:

1. The OS and its system libraries. `python:3.11-slim` is Debian bookworm with a known glibc, libstdc++, and OpenSSL. PyTorch wheels are compiled against specific glibc versions; a colleague on an older Linux distro can `pip install` the exact same `torch==2.4.1` and get an `ImportError: GLIBC_2.32 not found`. The container sidesteps that.

2. The Python interpreter itself. `requirements.txt` pins library versions; it does not pin Python. My initial run failed on Python 3.13 because `torch==2.4.1` has no wheel for it. The `FROM python:3.11-slim` line in the Dockerfile freezes this.

3. The execution layout. `COPY src/ ./src/` means `python -m src.main` always finds the package at the same path. Reproducing from a venv depends on the user's working directory and shell.

Of the three, the **Python interpreter pin** is the one that actually bit me during this homework — and it is invisible from `requirements.txt` alone.

03 One thing that broke, one thing that surprised you.

Broke. `train_test_split(train_size=5000, test_size=1000)` crashed because the SMS Spam Collection only has 5,572 messages. The slide suggested "5,000 train + 1,000 test" as a generic target across all three suggested datasets; with this particular one, the sum exceeds the corpus size by 428. Lesson: read the dataset before copying split sizes from a slide. I dropped train to 4,000.

Surprised. How lopsided the comparison was. I expected the transformer to lose by a few points, the way the slide's example numbers suggested (0.86 vs. 0.91 on sentiment data). Instead it lost by 50 percentage points because the pretraining domain was wrong. The headline metric on a leaderboard tells you nothing about whether a model fits *your* problem — that is what the test split on *your* data is for, and why running the comparison was worth doing instead of trusting the literature.